Instituto Tecnológico de Costa Rica
San Carlos Local Technological Campus
School of Computer Engineering

IC-6200 Artificial Intelligence

JarvisTEC

A voice and vision personal assistant that runs machine learning models

Group project

Team members:
Anthony Jafeth Arias Robleto — 2020007243
Randy Espinoza Downing — 2021031263
Kenneth Jhoan Araya Arguedas — 2024177221

Professor: Ing. Efrén Jiménez Delgado

First semester, 2026

Contents

1 Proposed solution **2**
- 1.1 Components . . . 2
- 1.2 Flow of an instruction . . . 3
- 1.3 Commands per model . . . 3

2 ML architectures **4**
- 2.1 Emotion recognition . . . 4
- 2.2 Speech recognition . . . 5
- 2.3 Networks trained from scratch (optional) . . . 5
- 2.4 Common architecture of the predictive models . . . 6

3 Justification and explanation of the developed models **7**
- 3.1 Wine quality . . . 7
- 3.2 Used car price . . . 7
- 3.3 Telecom customer churn . . . 7
- 3.4 Stroke risk . . . 7
- 3.5 Avocado price . . . 8
- 3.6 Bitcoin price . . . 8
- 3.7 Body fat percentage . . . 8
- 3.8 Movie recommendation . . . 8
- 3.9 Hepatitis C category . . . 8
- 3.10 Cirrhosis stage . . . 9

4 Analysis of the results **9**
- 4.1 Discussion . . . 10
- 4.2 Example cases used by the assistant . . . 10
- 4.3 Response times . . . 10
- 4.4 General limitations . . . 11

A Appendices **13**
- A.1 Repository and execution . . . 13
- A.2 Code structure . . . 13
- A.3 Datasets . . . 14
- A.4 Evaluation figures . . . 14
- A.5 Screenshots . . . 18

1 Proposed solution

JarvisTEC is a personal assistant that takes instructions by voice or text, watches the user's face through the camera and answers by running a machine learning model. The user says, for example, "JarvisTEC, wine quality"; the system turns the speech into text, identifies which of the ten models the instruction refers to, runs the model and answers with a sentence, which is also read aloud. When the camera is on, the system estimates the user's dominant emotion and adapts the tone of the answer and the color of the interface. The interface and the answers are in English; commands are understood in English and in Spanish.

1.1 Components

Web interface. A page written in HTML, CSS and JavaScript with no dependencies, styled as a reactor after the reference interface of the assignment. It captures camera frames (WebRTC), listens to the microphone, animates the reactor with the real microphone volume and reads the answer with the browser's speech synthesis. It lists the available commands and, for each model, shows a form generated automatically from the model's inputs, so the user can query the model with their own values.

REST API. A FastAPI service that exposes every resource of the system (Table 1).

Emotion agent. Takes an image and returns the dominant emotion and a probability per emotion. It runs locally in the service with pretrained neural networks: a face detector and two facial expression classifiers whose probabilities are averaged. It needs no API key or external service.

Speech agent. Turns the user's speech into text, in English, with either of two engines the user can switch between: the browser's speech recognition (the Web Speech API of Chrome and Edge) or a local model in the service (Vosk, an offline recognizer), which receives the recorded audio through the API. The service also keeps a Google Cloud Speech-to-Text provider, which is activated by configuring an API key.

Command router. Normalizes the text (lowercase, no accents or punctuation), tolerates variants of the wake word ("Jarvis TEC", "yarvis tek") and compares the text with the phrases of each model, first by exact match and then by fuzzy similarity.

Model orchestrator. The single entry point to the models: it validates the inputs against each model's schema, loads the trained model once, runs it and builds the answer. If one model is unavailable, the others keep working.

Ten machine learning models. Each has a training notebook that follows the six sections of the rubric, a trained artifact stored in the repository and an inference adapter.

Table 1: Resources of the REST API.

Method	Path	Description
GET	`/health`	API status, registered models and active providers
GET	`/models`	Metadata of every model: inputs, allowed values, default values and command phrases
GET	`/models/{name}`	Metadata of one model
POST	`/models/{name}/predict`	Runs a model with the given inputs; missing inputs take their default value
POST	`/command`	Takes the text of an instruction, routes it to the matching model and runs it
POST	`/emotion`	Takes an image and returns the dominant emotion
POST	`/transcribe`	Takes an audio clip (WAV) and returns its text and a confidence, with the local speech model or with Google

1.2 Flow of an instruction

1. The user presses the microphone button and speaks. With the browser engine, the browser transcribes the speech and shows the words as it recognizes them; with the local engine, the page records the audio and sends it to `/transcribe`.

2. The transcribed text is sent to `/command`. In parallel, if the camera is on, frames are sent to `/emotion`.

3. The router identifies the model. Three commands also take data from the sentence itself ("red wine quality", "organic avocado price", "recommend me a movie like Jurassic Park"); the remaining inputs come from the model's form or from its default values.

4. The orchestrator validates the inputs, runs the model and returns the result together with a sentence for the user.

5. The interface shows the answer, reads it aloud and prefixes an empathetic phrase according to the detected emotion.

1.3 Commands per model

Table 2 lists the main phrase of each model. Each model also accepts between three and six alternative phrases. A phrase matches when its words appear in the command as whole words, in order; small slips of the speech recognizer are forgiven only in long words and only when every word of the phrase matches, so a command about another topic ("JarvisTEC, gold price") reaches no model. Every model also has Spanish phrases ("JarvisTEC, calidad del vino", "precio del carro", "tipo de cirrosis"), matched by the same rules; a button in the interface tells the speech recognition which language to listen for, and the answers remain in English.

Table 2: Main instruction of each model.

Model	Instruction
Wine quality	“JarvisTEC, wine quality”
Used car price	“JarvisTEC, car price”
Customer churn	“JarvisTEC, customer churn”
Stroke risk	“JarvisTEC, stroke risk”
Avocado price	“JarvisTEC, avocado price”
Bitcoin price	“JarvisTEC, bitcoin price”
Body fat	“JarvisTEC, body fat”
Movie recommendation	“JarvisTEC, recommend a movie”
Hepatitis C category	“JarvisTEC, hepatitis type”
Cirrhosis stage	“JarvisTEC, cirrhosis stage”

2 ML architectures

2.1 Emotion recognition

The interface captures three video frames, 150 ms apart, and sends each one to `POST /emotion`; the scores are averaged so that a blink or a half-formed expression weighs less. The service processes each frame locally with OpenCV in two stages. First, the YuNet face detector [35], a lightweight convolutional network, locates the faces in the frame; the largest one, which corresponds to the user in front of the camera, is kept and cropped to a square. Then two networks classify the expression: FER+ [1], a convolutional network trained on FER-2013 with labels corrected by several annotators, which takes the face as a 64×64 grayscale image; and HSEmotion [29], an EfficientNet-B0 trained on AffectNet, which takes the face as a 224×224 color image. Both produce a probability for eight emotions (neutral, happiness, surprise, sadness, anger, disgust, fear and contempt); the service averages the two distributions and reports the most probable emotion. If no face is detected, the API answers with a clear error for the user.

The three networks are used pretrained, in ONNX format. To justify the combination, the classifiers were evaluated on the 7178 test faces of FER-2013 with their original labels (Table 3). FER+ alone classifies most sad and angry faces as neutral; HSEmotion recognizes them better but confuses more neutral faces; the average reaches the highest accuracy and recovers part of the negative emotions without losing the neutral or happy ones. The FER-2013 images are 48×48 grayscale pictures, outside the training domain of HSEmotion, so its figures are a lower bound of what to expect from a camera. Even so, subtle or posed expressions remain difficult: sadness and fear are recognized in fewer than a third of the test faces. As an alternative, the service keeps a Google Cloud Vision provider, which is activated by configuring an API key.

Table 3: Emotion classifiers on the 7178 test faces of FER-2013: accuracy, macro-averaged recall and recall of four emotions.

Model	Accuracy	Macro recall	Sadness	Anger	Fear	Neutral
FER+	0.565	0.485	0.241	0.381	0.062	0.912
HSEmotion	0.560	0.523	0.407	0.511	0.384	0.564
Average of both (in use)	0.601	0.551	0.316	0.443	0.194	0.857

The combination was chosen on this same test set, so Table 3 describes it rather than validating it independently. The team tested the complete flow with a real camera in Chrome.

2.2 Speech recognition

Two engines are available and the user switches between them with a button. **Browser engine.** The interface uses the browser's Web Speech API with language `en-US`. The browser captures the audio, transcribes it with its vendor's speech service and delivers partial and final results; only the final text reaches the backend, where it is sent to `/command`. It works in Chrome and Edge and requires an internet connection. The team tested this flow with a real microphone in Chrome.

Local engine. The service runs Vosk, an offline recognizer built on the Kaldi toolkit [24], with its small English and Spanish models (about 40 MB each), stored in the repository. The page records the microphone as 16-bit mono WAV at 16 kHz and sends it to `POST /transcribe`, which returns the text and the mean confidence of its words. It needs no internet connection and works in any browser; if the browser engine fails, the page switches to this one by itself. The English model was checked with 15 commands spoken by a synthetic voice: every transcription reached the intended model, in about 0.7 s per command. The Spanish model was not checked with speech, and neither has been measured with real voices; the word error rate published for the model is about 10 %, so it is less accurate than the browser engine. Because the transcription of the wake word varies ("Jarvis TEC", "Jarvis tech", "yarvis tek"), the command router tolerates those variants. Neither engine needs an API key or a billing account. The service also includes a Google Cloud Speech-to-Text provider, activated by configuring an API key.

2.3 Networks trained from scratch (optional)

The engines above are pretrained by third parties. To have models of our own for the two agents, two convolutional networks were trained from random initialization, without pretrained weights, with PyTorch on public datasets, and exported to ONNX so that the service runs them with OpenCV like the others. The pretrained engines remain the default; each network is enabled with one setting in the `.env` file (`EMOTION_MODEL=scratch` and `SPEECH_PROVIDER=scratch`), with no switch in the interface. Their notebooks (`training/emotion_cnn/` and `training/speech_commands/`) follow the same six sections as the predictive models.

Emotion. A VGG-style network [30] of four convolutional blocks (4.8 million parameters) classifies a 48×48 grayscale face into seven emotions. It was trained on the 28 709 training images of FER-2013 [10] (10 % held out for validation) with class weights, label smoothing and data augmentation, and evaluated once on the 7178 test images (Table 4). It surpasses the pretrained FER+ and HSEmotion on the same images and lies within the range reported for networks trained from scratch on this dataset. Inside the application, where the face is cropped by YuNet instead of by the dataset, it reaches 0.643 accuracy on 420 test faces placed in a camera-sized frame, against 0.561 for the average of FER+ and HSEmotion. FER-2013 labels are noisy and 568 test images also appear in the training set, a known issue of the dataset.

Speech. No public dataset contains the JarvisTEC phrases, so the network is a keyword spotter [26] trained on Speech Commands [33]: 105 829 one-second recordings of 35 English words by 2618 speakers. It classifies a log-mel spectrogram into twelve classes: the ten digits, any other word and silence. In the application the user says the number of a model ("JarvisTEC, three" runs the third model); a one-second window slides over the recording and the most confident digit is taken. Training used background noise, time shifts and SpecAugment [21]. On the official test speakers, never heard in training, it reaches 0.976 accuracy (Table 4); on 400 simulated recordings

of a word followed by a digit with background noise, it recognizes the right digit in 89 % of the cases, misses it in 10 % and chooses a wrong one in under 1 %. It does not understand sentences, Spanish or data inside the command, which is why Vosk stays the default.

Table 4: Networks trained from scratch on the test sets of their datasets, compared with their baselines.

Agent	Model	Accuracy	Macro F1
Emotion (FER-2013)	Most frequent class	0.247	0.057
	PCA + logistic regression	0.335	0.297
	FER+ (pretrained)	0.565	0.467
	HSEmotion (pretrained)	0.560	0.507
	CNN from scratch	**0.686**	**0.660**
Speech (Speech Commands)	Most frequent class	0.154	0.022
	Average spectrum + logistic regression	0.600	0.618
	CNN from scratch	**0.976**	**0.979**

2.4 Common architecture of the predictive models

All the models follow the same process, documented in one notebook per model:

1. **Problem analysis:** type of task, who it is useful for and success metric.
2. **Data understanding:** variables, missing values, duplicates and excluded columns.
3. **Data exploration:** distributions, correlations and the findings that justify the preprocessing decisions.
4. **Model:** the preprocessing (imputation, scaling, encoding of categories) lives inside a scikit-learn `Pipeline` [22], so the service receives raw data and there is no information leakage. The two time-series models are the exception in one respect: their lagged features are built outside the `Pipeline` by a single function implemented identically in the notebook and in the service; and the recommender is a lookup table rather than a `Pipeline`. A baseline is compared with several candidate models by cross-validation on the training set only, and the hyperparameters of the selected model are tuned with a search on that same set.
5. **Evaluation:** the chosen model is evaluated on a test set that was not used for training, selection or tuning (20 % of the data; the most recent part in the time series), always next to the baseline. Where a notebook also scores another model on the test set, it is for reference only and did not influence a choice.
6. **Conclusion:** result, limitations and future improvements.

The random seed is fixed (42) and every notebook can be re-executed to regenerate its model; the team re-executed the ten notebooks and obtained the same metric files. Table 5 summarizes the architecture chosen for each problem.

Table 5: Architecture chosen for each model.

Model	Task	Architecture
Wine	Classification (3 classes)	Random forest with balanced class weights
Car price	Regression	Gradient boosting on the logarithm of the price
Churn	Binary classification	Gradient boosting with a tuned decision threshold
Stroke	Binary classification	Random forest with a tuned decision threshold
Avocado	Time series	Ridge regression on lagged prices
Bitcoin	Time series	Random forest on logarithmic returns
Body fat	Regression	Ridge regression with two measurements
Movies	Recommendation	Item-based collaborative filtering
Hepatitis C	Classification (4 classes)	Random forest with balanced class weights
Cirrhosis	Classification (4 stages)	Random forest with balanced class weights

3 Justification and explanation of the developed models

3.1 Wine quality

Problem. Classify the sensory quality of a wine as low, medium or high from eleven physicochemical measurements and the wine type. The data are those of Cortez et al. [3], who modeled quality from these same variables. **Data.** 6497 wines; 1168 duplicated rows were removed before splitting. The original score (3 to 9) was grouped into three classes because the extreme scores have almost no examples. **Model.** Among logistic regression, random forest [2] and gradient boosting [8], the random forest had the best macro F1 in cross-validation. The exploration showed weak, non-linear relationships between chemistry and quality, which an ensemble of trees captures without assuming linearity. Balanced class weights compensate for the high class having half as many examples [12].

3.2 Used car price

Problem. Estimate the resale price of a vehicle from its year, the price of the same vehicle when new, the kilometers driven, the fuel, the transmission and the type of seller, as in the work of Pudaruth [25] and Gegic et al. [9]. **Data.** 301 listings from the Indian market, with prices in lakh of rupees. **Model.** The price is right-skewed and depreciation acts as a percentage of the new price, so a gradient boosting model was trained on the logarithm of the price. A linear regression on the logarithm was discarded because it extrapolates to absurd values for very expensive vehicles.

3.3 Telecom customer churn

Problem. Predict whether a customer will leave the company, a problem studied by Verbeke et al. [32] and Huang et al. [15]. **Data.** 7043 customers, 26.5 % of whom churned. **Model.** Gradient boosting with the decision threshold tuned on the training set (0.30 instead of 0.50), because missing a customer who leaves costs the company more than contacting one who stays. Logistic regression was practically tied in cross-validation.

3.4 Stroke risk

Problem. Estimate the probability of a stroke from age, hypertension, heart disease, glucose, body mass index and habits, risk factors recognized in the literature [34, 20]. **Data.** 5110 patients

and only 4.9 % positive cases, so accuracy is misleading and recall, precision and the areas under the ROC and precision-recall curves are used instead [27]. **Model.** A random forest without class weights (the weights inflated the probabilities without improving the ranking). The threshold (0.10) was chosen from the cross-validated predictions on the training set. The result is presented as an estimate with three risk levels, never as a diagnosis.

3.5 Avocado price

Problem. Forecast the average price of the following week. **Data.** Weekly prices from 2015 to 2018 for 54 regions of the United States and two types (conventional and organic). **Model.** Ridge regression [13] on the prices of the four previous weeks, their moving average, the week of the year and the type. The split is chronological and the model is compared with the naive forecast ("same as last week"), the standard reference in time series [16]. Same-week variables (volume, bags) were excluded because they are not known when the forecast is made.

3.6 Bitcoin price

Problem. Forecast the closing price of the following day. **Data.** Daily prices from April 2013 to July 2017. **Model.** Because the price grows more than twenty-fold over the period and trees cannot extrapolate levels, the model predicts the next-day logarithmic return from lagged returns and rolling statistics and converts it back to a price. The comparison with the naive forecast is essential: the efficient market hypothesis [6] anticipates that it is very hard to beat, and the prediction of the Bitcoin price with machine learning has been studied, among others, by McNally et al. [19].

3.7 Body fat percentage

Problem. Estimate the body fat percentage from simple measurements instead of underwater weighing [17, 23]. **Data.** 252 men. Body density was excluded because the body fat percentage is computed directly from it with Siri's equation [31]; using it would leak the target. Six rows with inconsistent values were removed. **Model.** A forward selection showed that two measurements, the abdomen circumference and the weight, give almost the same error as all thirteen. With so little data, ridge regression outperformed the tree ensembles.

3.8 Movie recommendation

Problem. Recommend movies similar to one the user liked. **Data.** 105,339 ratings of 10,329 movies, of the MovieLens kind [11]. **Model.** Item-based collaborative filtering [28]: two movies are similar if the same users rate them; the cosine similarity is shrunk when few users are in common. It was compared with recommending the most popular movies, with a genre-based similarity and with latent factors [18]. The evaluation uses ranking metrics on held-out ratings of each user [4].

3.9 Hepatitis C category

Problem. Classify a patient as a healthy blood donor, hepatitis, fibrosis or cirrhosis from ten laboratory values, with the data of Hoffmann et al. [14]. **Data.** 615 patients, about 87 % of whom are healthy donors; each stage of the disease has between 21 and 30 cases. The seven "suspect donor" rows were removed because their label is not confirmed. **Model.** A random forest with balanced class weights, chosen by macro F1 with repeated stratified cross-validation.

3.10 Cirrhosis stage

Problem. Estimate the histologic stage (1 to 4) of the liver disease from clinical and laboratory data of the Mayo Clinic trial on primary biliary cirrhosis [7, 5]. **Data.** 412 patients with a known stage. The follow-up time and the outcome were excluded because they are only known after the stage is assessed. **Model.** A random forest with balanced class weights. The four stages were kept because the assignment asks for them, although they are hard to separate with these data.

4 Analysis of the results

All the values come from the evaluation of the chosen model on the test set of each notebook. Table 6 summarizes the classification models, Table 7 the regression and time-series models, and Table 8 the recommender. The evaluation figure of every model is in Appendix A.4.

Table 6: Results of the classification models on the test set.

Model	Cases	Model	Baseline
Wine	1066	Macro F1 0.590; accuracy 0.591	Macro F1 0.203
Churn	1409	Churn F1 0.628; recall 0.770; precision 0.529; ROC-AUC 0.846	Churn F1 0
Stroke	1022	ROC-AUC 0.828; recall 0.700; precision 0.183	ROC-AUC 0.500
Hepatitis C	122	Macro F1 0.588; balanced accuracy 0.589	Macro F1 0.234
Cirrhosis	83	Macro F1 0.443; accuracy 0.506	Macro F1 0.136; accuracy 0.373

Table 7: Results of the regression and time-series models on the test set.

Model	Model	Reference
Car price (lakh)	RMSE 1.79; MAE 0.87; R^2 0.876	Mean: RMSE 5.08
Body fat (points)	RMSE 4.27; MAE 3.23; R^2 0.672	Mean: RMSE 7.45
Avocado, 108 series (USD)	RMSE 0.153; MAPE 7.7 %	Naive: RMSE 0.164
Avocado, national series (USD)	RMSE 0.079; MAPE 4.6 %	Naive: RMSE 0.076
Bitcoin (USD)	RMSE 69.98; MAPE 2.44 %	Naive: RMSE 69.73

Table 8: Movie recommender: NDCG@10 on the test set (655 users).

Approach	One seed movie	Full history
Most popular	0.165	0.165
Item-based collaborative filtering	0.140	0.269
Latent factors (SVD)	0.086	0.276
Genre similarity	0.006	0.006

4.1 Discussion

- **Models with a clear signal.** The car price (R^2 0.88, median error of 12 % of the price), customer churn (ROC-AUC 0.85) and wine (macro F1 almost three times that of the baseline) learn a real relationship. For the car price, the new price and the year explain almost everything: the model learned depreciation by age.
- **Medical models: useful as guidance, not as a diagnosis.** The stroke model detects 70 % of the cases, but only 18 % of its alerts are correct, and age alone ranks the patients slightly better (ROC-AUC 0.835 against 0.828); what the model adds is a probability and a better precision-recall area. The hepatitis model separates a healthy donor from a patient well (it detects 13 of 15), but it gets the exact stage right in 7 of those 13, with only 4 to 6 test patients per stage. The cirrhosis model gets the exact stage right in half of the cases, against 37 % for the baseline that always answers the most frequent stage; on the looser criterion of being at most one stage off, that baseline does better than the model (95 % against 89 %), because the most frequent stage is in the middle of the scale. Stages 2 and 3 are confused with each other. For that reason the spoken answers of these models state that they are estimates.
- **Time series: the naive forecast is hard to beat.** For the avocado, the model beats the naive forecast over the 108 series as a whole, but on the national series, the one the assistant answers with, it is tied. For Bitcoin the model does not beat the naive forecast, which is consistent with the efficient market hypothesis [6]. Both results are reported as they are; presenting these models as superior would be incorrect.
- **Recommender.** With the full history of a user, collaborative filtering clearly beats recommending the most popular movies (the full-history row of Table 8 uses a neighbourhood configuration tuned for that scenario, which differs from the table the assistant serves). In the scenario the assistant serves, with a single seed movie, popularity reaches a higher NDCG. Collaborative filtering is served because it is the best approach whose answer depends on the given movie and because it covers 20 % of the catalogue instead of always repeating the same titles. The evaluation removes the movies each user already rated, which the assistant cannot do because it does not know the user.
- **Body fat.** Two measurements give a typical error of 3.2 percentage points. The data only include men, so the model must not be used for women or children.

4.2 Example cases used by the assistant

A command without data answers for an example case. For the wine, churn and cirrhosis models the example is a real case taken from the held-out test set that the model classifies clearly (a high-quality wine, a customer who left, a stage-4 patient), so the demonstration shows a confident answer; for the other models it is a typical case built from the medians of the training data. These examples only illustrate the service: they play no part in the metrics above, and the probability spoken with an answer is the model's estimate for that case, not the model's accuracy.

4.3 Response times

Table 9 shows the response times of the API measured on the development machine, without a GPU, with `scripts/measure_response_times.py`: the median of 20 consecutive requests after the model is loaded. Models are loaded on their first request; the slowest first request is the wine model (about 1.4 s), and the others load in under 0.4 s.

Table 9: Median response time of the API once each model is loaded.

Request	Time (ms)	Request	Time (ms)
Wine	22	Bitcoin	13
Car price	5	Body fat	2
Churn	11	Movies	1
Stroke	23	Hepatitis C	13
Avocado	3	Cirrhosis	22
Emotion (one frame)	34	Command (router + wine)	22

4.4 General limitations

Several datasets are small (body fat, car price, cirrhosis, hepatitis), so their metrics carry wide uncertainty. The price data are historical (avocado until 2018, Bitcoin until 2017), and the assistant says so in its answer. Car prices are in Indian rupees. No medical model was clinically validated. The pretrained emotion networks were evaluated on a public benchmark, not on camera frames of the intended users.

References

[1] E. Barsoum, C. Zhang, C. Canton Ferrer and Z. Zhang, "Training deep networks for facial expression recognition with crowd-sourced label distribution," in *Proc. 18th ACM Int. Conf. on Multimodal Interaction (ICMI)*, 2016, pp. 279–283.

[2] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[3] P. Cortez, A. Cerdeira, F. Almeida, T. Matos and J. Reis, "Modeling wine preferences by data mining from physicochemical properties," *Decision Support Systems*, vol. 47, no. 4, pp. 547–553, 2009.

[4] P. Cremonesi, Y. Koren and R. Turrin, "Performance of recommender algorithms on top-N recommendation tasks," in *Proc. ACM RecSys*, 2010, pp. 39–46.

[5] E. R. Dickson, P. M. Grambsch, T. R. Fleming, L. D. Fisher and A. Langworthy, "Prognosis in primary biliary cirrhosis: Model for decision making," *Hepatology*, vol. 10, no. 1, pp. 1–7, 1989.

[6] E. F. Fama, "Efficient capital markets: A review of theory and empirical work," *The Journal of Finance*, vol. 25, no. 2, pp. 383–417, 1970.

[7] T. R. Fleming and D. P. Harrington, *Counting Processes and Survival Analysis*. Wiley, 1991.

[8] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[9] E. Gegic, B. Isakovic, D. Keco, Z. Masetic and J. Kevric, "Car price prediction using machine learning techniques," *TEM Journal*, vol. 8, no. 1, pp. 113–118, 2019.

[10] I. J. Goodfellow et al., "Challenges in representation learning: A report on three machine learning contests," in *Proc. Int. Conf. on Neural Information Processing (ICONIP)*, 2013, pp. 117–124.

[11] F. M. Harper and J. A. Konstan, "The MovieLens datasets: History and context," *ACM Transactions on Interactive Intelligent Systems*, vol. 5, no. 4, 2015.

[12] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009.

[13] A. E. Hoerl and R. W. Kennard, "Ridge regression: Biased estimation for nonorthogonal problems," *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970.

[14] G. Hoffmann, A. Bietenbeck, R. Lichtinghagen and F. Klawonn, "Using machine learning techniques to generate laboratory diagnostic pathways—a case study," *Journal of Laboratory and Precision Medicine*, vol. 3, p. 58, 2018.

[15] B. Huang, M. T. Kechadi and B. Buckley, "Customer churn prediction in telecommunications," *Expert Systems with Applications*, vol. 39, no. 1, pp. 1414–1425, 2012.

[16] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. OTexts, 2021.

[17] R. W. Johnson, "Fitting percentage of body fat to simple body measurements," *Journal of Statistics Education*, vol. 4, no. 1, 1996.

[18] Y. Koren, R. Bell and C. Volinsky, "Matrix factorization techniques for recommender systems," *Computer*, vol. 42, no. 8, pp. 30–37, 2009.

[19] S. McNally, J. Roche and S. Caton, "Predicting the price of Bitcoin using machine learning," in *Proc. 26th Euromicro Int. Conf. on Parallel, Distributed and Network-based Processing (PDP)*, 2018, pp. 339–343.

[20] M. J. O'Donnell et al., "Global and regional effects of potentially modifiable risk factors associated with acute stroke in 32 countries (INTERSTROKE): a case-control study," *The Lancet*, vol. 388, no. 10046, pp. 761–775, 2016.

[21] D. S. Park, W. Chan, Y. Zhang, C.-C. Chiu, B. Zoph, E. D. Cubuk and Q. V. Le, "SpecAugment: A simple data augmentation method for automatic speech recognition," in *Proc. Interspeech*, 2019, pp. 2613–2617.

[22] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[23] K. W. Penrose, A. G. Nelson and A. G. Fisher, "Generalized body composition prediction equation for men using simple measurement techniques," *Medicine and Science in Sports and Exercise*, vol. 17, no. 2, p. 189, 1985.

[24] D. Povey et al., "The Kaldi speech recognition toolkit," in *Proc. IEEE Workshop on Automatic Speech Recognition and Understanding (ASRU)*, 2011.

[25] S. Pudaruth, "Predicting the price of used cars using machine learning techniques," *International Journal of Information & Computation Technology*, vol. 4, no. 7, pp. 753–764, 2014.

[26] T. N. Sainath and C. Parada, "Convolutional neural networks for small-footprint keyword spotting," in *Proc. Interspeech*, 2015, pp. 1478–1482.

[27] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLoS ONE*, vol. 10, no. 3, e0118432, 2015.

[28] B. Sarwar, G. Karypis, J. Konstan and J. Riedl, "Item-based collaborative filtering recommendation algorithms," in *Proc. 10th Int. Conf. on World Wide Web*, 2001, pp. 285–295.

[29] A. V. Savchenko, L. V. Savchenko and I. Makarov, "Classifying emotions and engagement in online learning based on a single facial expression recognition neural network," *IEEE Transactions on Affective Computing*, vol. 13, no. 4, pp. 2132–2143, 2022.

[30] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2015.

[31] W. E. Siri, "The gross composition of the body," *Advances in Biological and Medical Physics*, vol. 4, pp. 239–280, 1956.

[32] W. Verbeke, K. Dejaeger, D. Martens, J. Hur and B. Baesens, "New insights into churn prediction in the telecommunication sector: A profit driven data mining approach," *European Journal of Operational Research*, vol. 218, no. 1, pp. 211–229, 2012.

[33] P. Warden, "Speech Commands: A dataset for limited-vocabulary speech recognition," arXiv:1804.03209, 2018.

[34] P. A. Wolf, R. B. D'Agostino, A. J. Belanger and W. B. Kannel, "Probability of stroke: A risk profile from the Framingham Study," *Stroke*, vol. 22, no. 3, pp. 312–318, 1991.

[35] W. Wu, H. Peng and S. Yu, "YuNet: A tiny millisecond-level face detector," *Machine Intelligence Research*, vol. 20, pp. 656–665, 2023.

A Appendices

A.1 Repository and execution

Repository: `https://github.com/Ran147/proyecto_AI_1`. To run the system:

1. Create a virtual environment and install `requirements.txt`.
2. Start the service: `uvicorn backend.main:app`.

The trained models, the emotion networks and the speech model are stored in the repository (`artifacts/`), so nothing has to be trained or downloaded to run it. To retrain a model, `python scripts/download_datasets.py` fetches the datasets and each notebook in `training/` regenerates its model. The interface is served at `http://localhost:8000` and the interactive API documentation at `/docs`.

A.2 Code structure

- `backend/`: API, orchestrator, command router, emotion and speech services, and one inference package per model.
- `frontend/`: web interface.

- `training/`: one notebook and one model card per model.
- `artifacts/`: the trained models, the emotion networks and the speech model.
- `scripts/`: dataset and model downloads, evaluation of the emotion models and response-time measurement.
- `tests/`: automated tests of the API, the router, the emotion service and each model.

A.3 Datasets

Model	Source
Wine	Kaggle, `rajyellow46/wine-quality`
Car price	GitHub, `amankharwal/Website-data` (`car data.csv`)
Churn	GitHub, `IBM/telco-customer-churn-on-icp4d`
Stroke	Kaggle, `fedesoriano/stroke-prediction-dataset`
Avocado	Kaggle, `neuromusic/avocado-prices`
Bitcoin	Kaggle, `team-ai/bitcoin-price-prediction`
Body fat	Kaggle, `fedesoriano/body-fat-prediction-dataset`
Movies	Google Drive file given in the assignment
Hepatitis C	Kaggle, `fedesoriano/hepatitis-c-dataset`
Cirrhosis	Kaggle, `fedesoriano/cirrhosis-prediction-dataset`
Emotion (evaluation only)	Kaggle, `msambare/fer2013`

A.4 Evaluation figures

Each figure is the evaluation plot produced by the corresponding notebook on its test set.

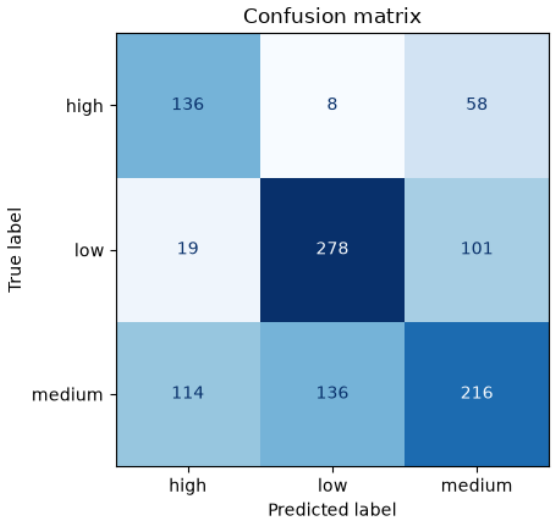


Figure 1: Wine quality: confusion matrix on the test set.

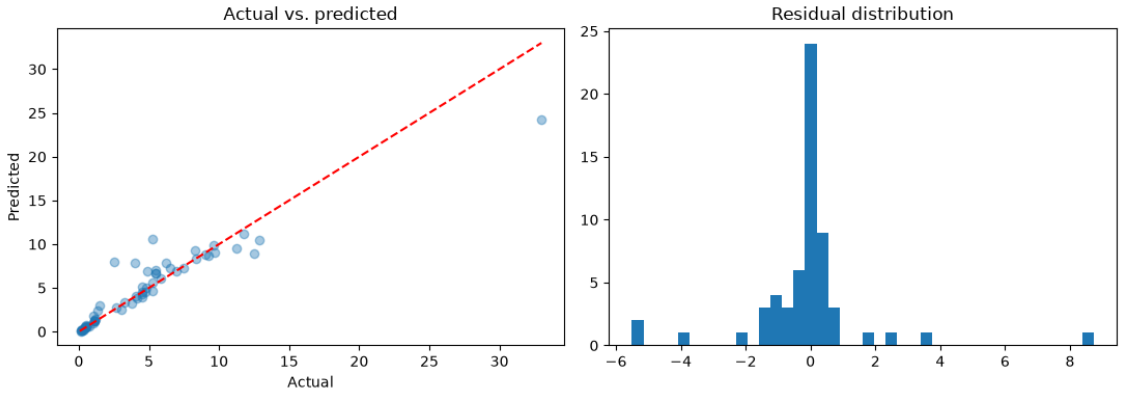


Figure 2: Used car price: actual against predicted price and distribution of the residuals.

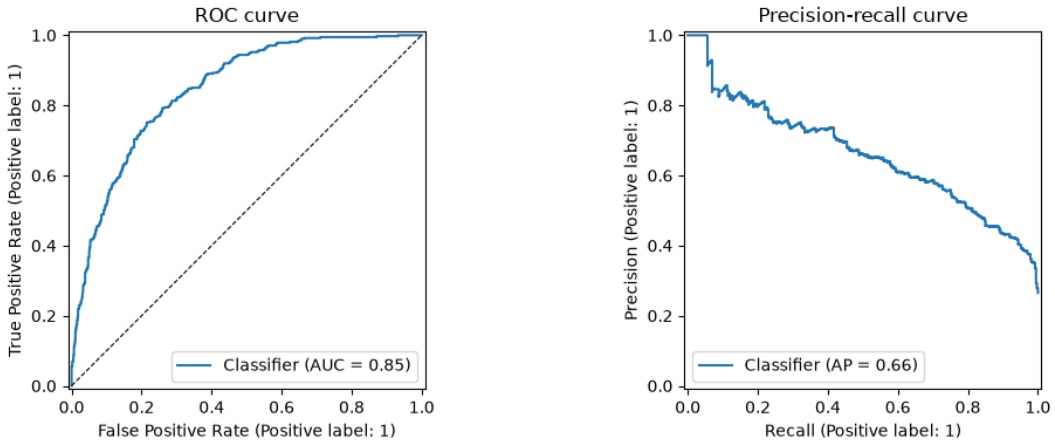


Figure 3: Customer churn: ROC and precision-recall curves on the test set.

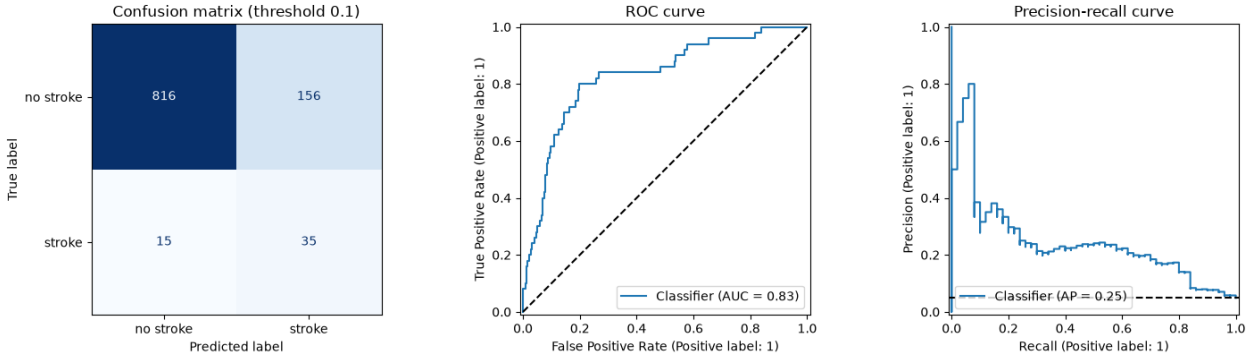


Figure 4: Stroke risk: confusion matrix at the chosen threshold, ROC curve and precision-recall curve.

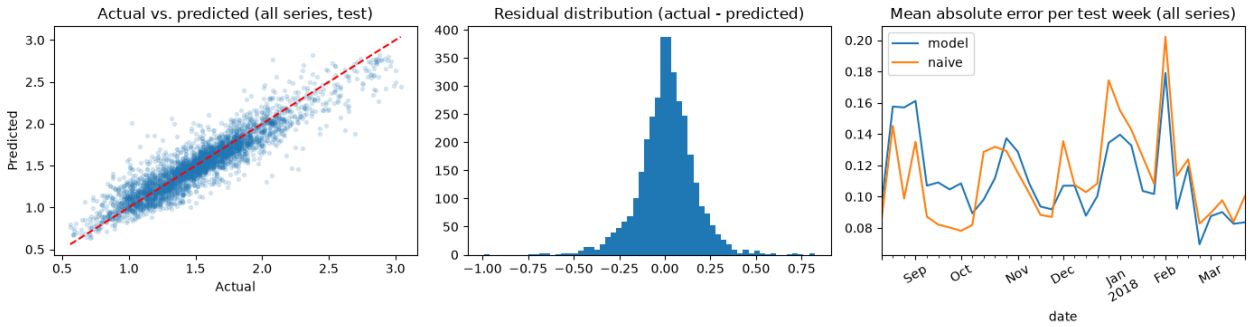


Figure 5: Avocado price: actual against predicted price, residuals and weekly error of the model and the naive forecast.

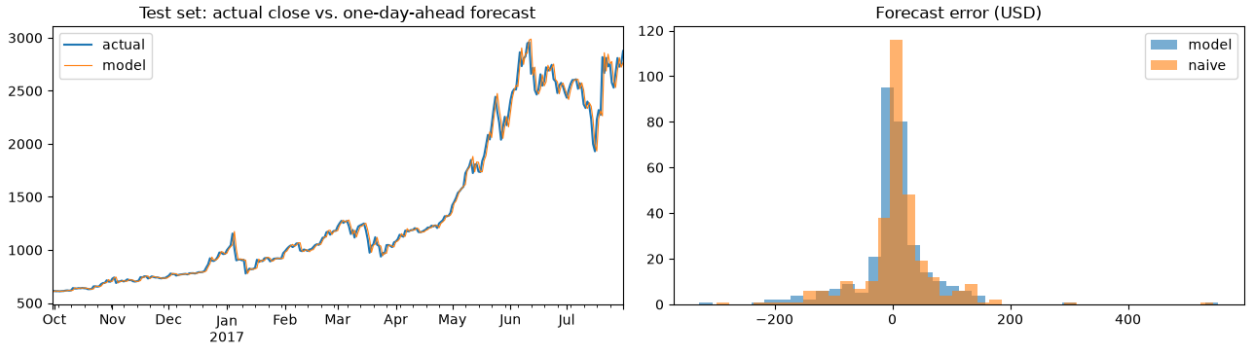


Figure 6: Bitcoin price: actual close against the one-day-ahead forecast over the test period, and forecast error.

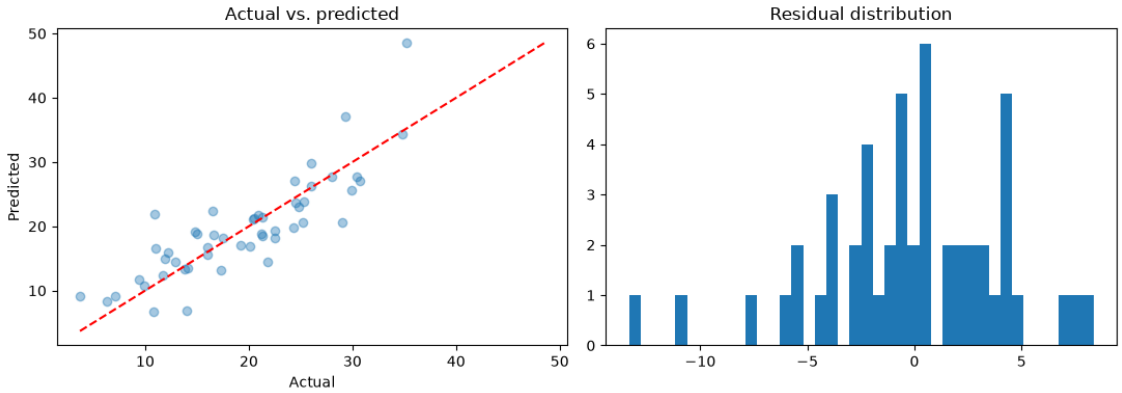


Figure 7: Body fat: actual against predicted percentage and distribution of the residuals.

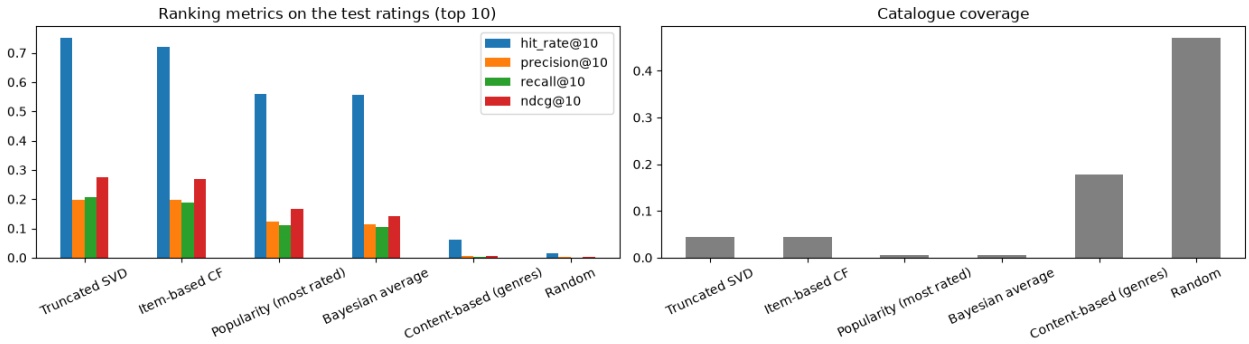


Figure 8: Movie recommender: ranking metrics of every approach on the test ratings and catalogue coverage.

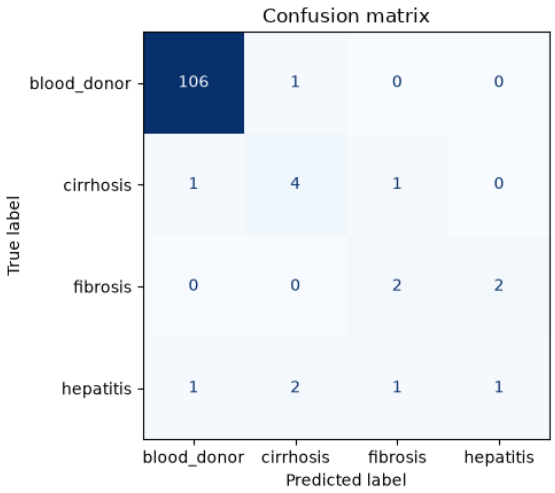


Figure 9: Hepatitis C category: confusion matrix on the test set.

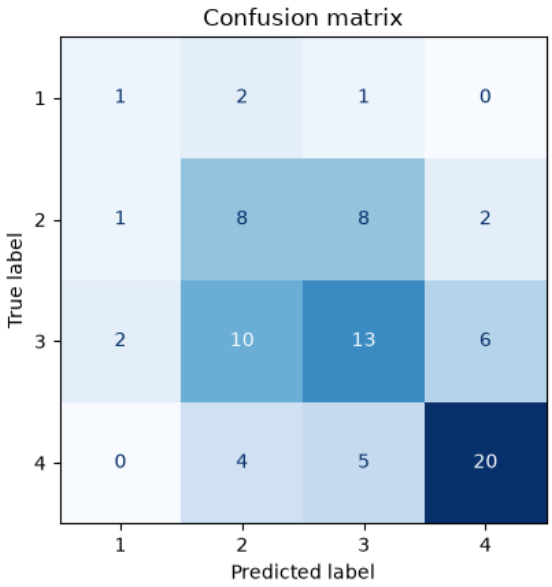


Figure 10: Cirrhosis stage: confusion matrix on the test set.

A.5 Screenshots

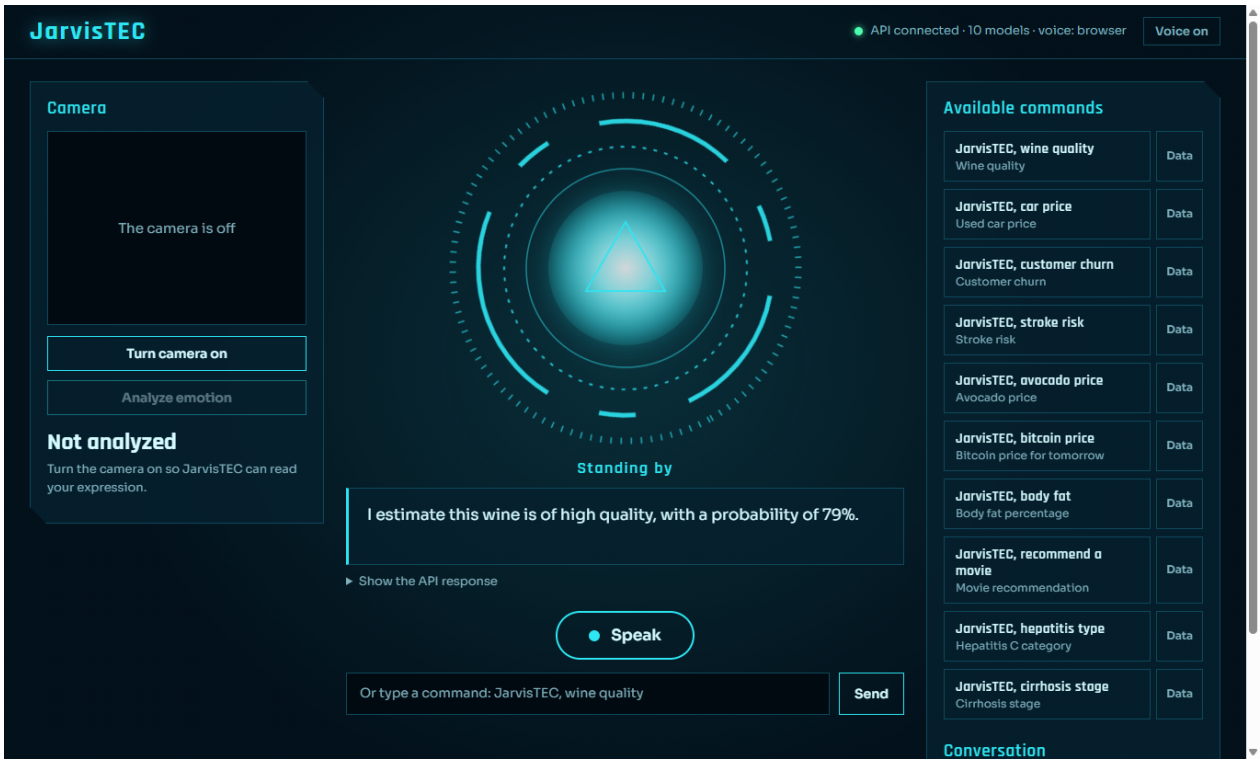


Figure 11: The interface after the command “JarvisTEC, wine quality”: camera panel, assistant answer and list of commands.

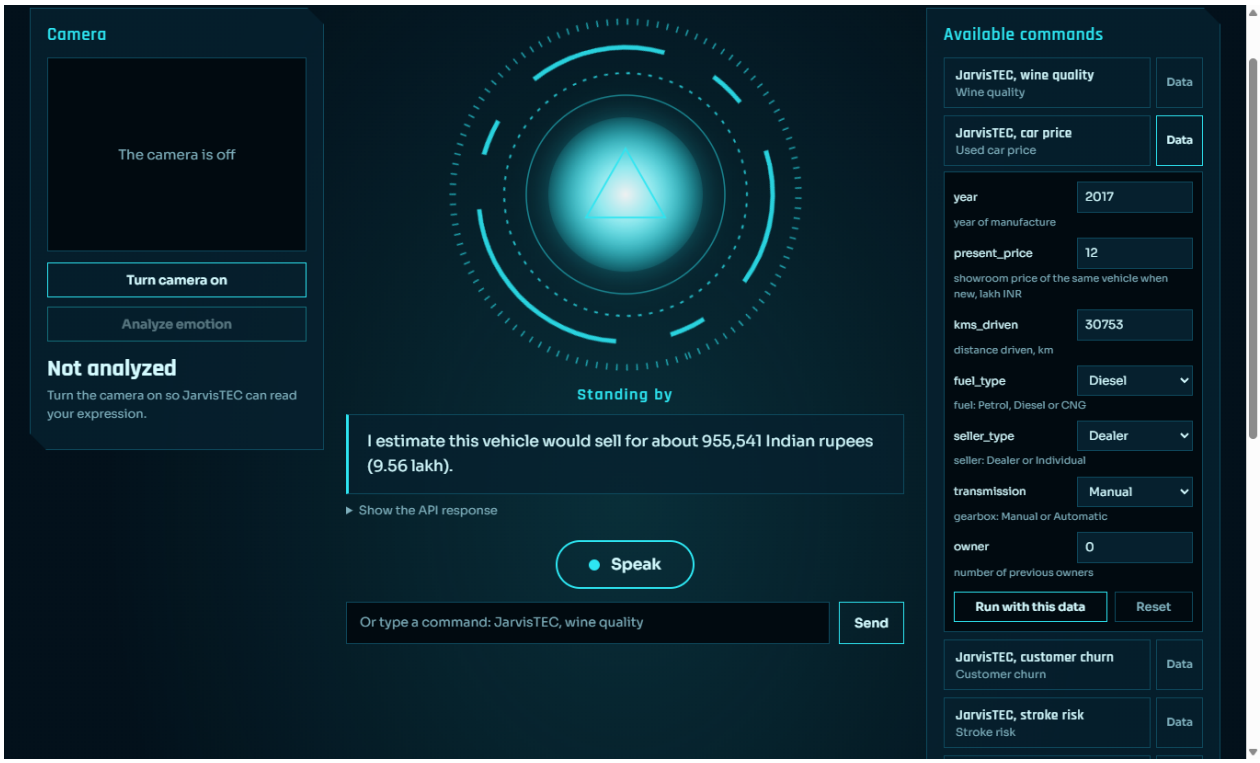


Figure 12: The data form of the car price model, generated from the model's input schema, and the answer for the edited values.

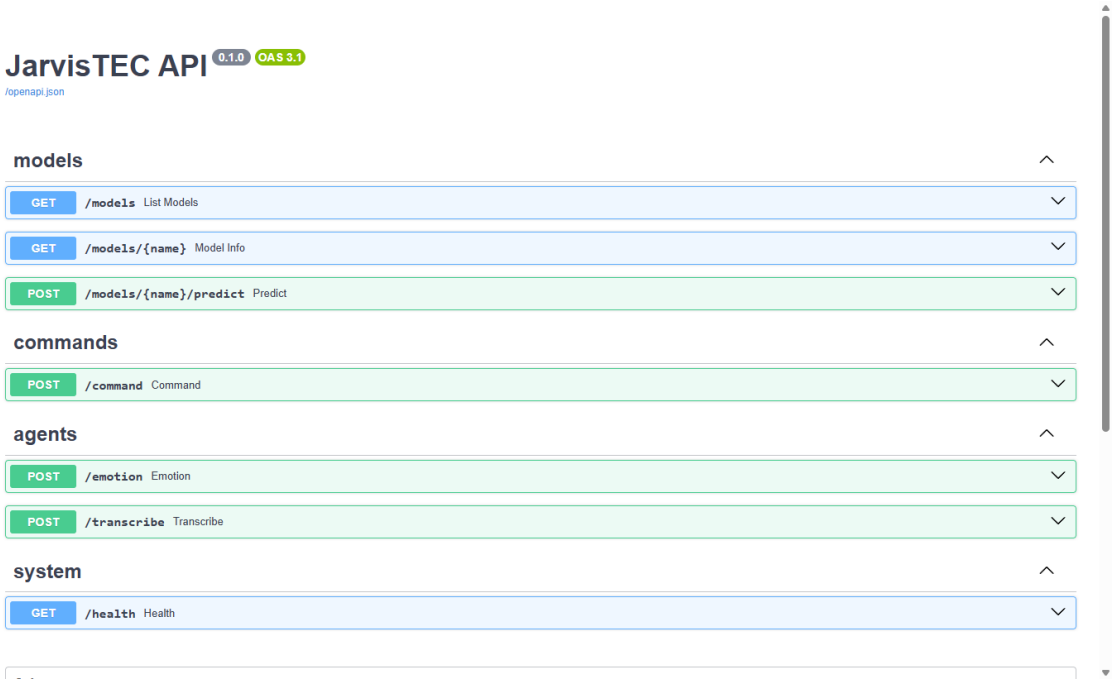


Figure 13: Interactive documentation of the REST API (`/docs`).